

C++
(the rest of chapter 7)

Passing Objects to Functions

Constant Reference Parameters

Returning an Object From A Function

Copy Constructors

Object Composition

Structures

Passing Objects to Functions

- Passing an object by value makes a copy of the object. This means the copy constructor is called and all the values of the data members are copied into the new object.

```
void showValues(InventoryItem item) { ...
```

- This can slow down a program's execution time, especially if it is large or has many members.
- Any mutator functions called in the function will change only the copied object, not the original object.

Constant Reference Parameters

- Passing an object by reference is faster because it doesn't have to make a copy of all the object's members.
- Passing an object by reference is preferable.
- To help guard against changing the private data members of the object, the reference parameter should be passed as a constant reference:

```
void showValues(const InventoryItem &item) { ...
```

- This means that a reference to the object is passed in to the function but it cannot call any of the mutator functions.
- It also cannot call any accessor functions unless the word `const` is also placed after the function name:

```
double getPrice() const { ...
```

Returning an Object From A Function

- When a function returns an object, it may take in an object as an argument, or create a local instance of an object, then modify or set its data member values, and then return the object.
- This involves a copy constructor (if object sent in as an argument), or a regular constructor (if creating a local object); and then also a destructor at the end of the function when the object goes out of scope.

Copy Constructors

- A copy constructor is called when:
 1. An object is initialized from an object of the same class
 2. An object is passed by value to a function
 3. An object is returned using a **return** statement from a function

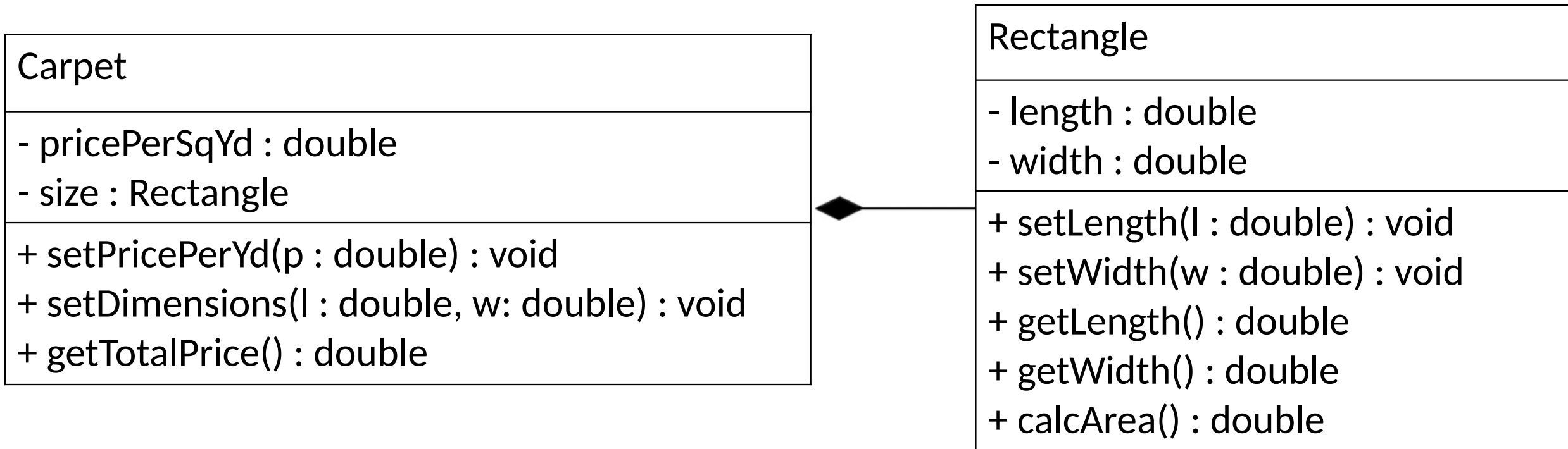
Object Composition

- It is possible for a class to have a member variable that is an instance of another class.
- Example from book:

```
class Rectangle {  
    private:  
        double length;  
        double width;  
    public:  
        void setLength(double);  
        void setWidth(double);  
        double getLength();  
        double getWidth();  
        double calcArea();  
};
```

```
class Carpet {  
    private:  
        double pricePerSqYd;  
        Rectangle size;  
    public:  
        void setPricePerYd(double p);  
        void setDimensions(double l, double w);  
        double getTotalPrice();  
};
```

Object Composition – “has a” relationship



C++ Structures

- unlike C structures, structures in C++
 - Can have member functions as well as data members (though usually don't)
 - Can initialize data member in structure definition
 - Do not need to use the word "struct" along with the name when declaring
 - Can have constructors
 - Data members normally do not use access modifiers, but default is public

```
struct Student {  
    int roll = 0;  
    Student(int x) { roll = x; }  
};  
  
int main() {  
    Student s(2);  
}
```